COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S1
DATE PERFORMED:	October 15, 2026	**DATE SUBMITTED:**	October 19, 2026
NAME:	De Leon, Benj Federic C. Mendoza, Ryo Shin P.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 13.1 ALGORITHMS STRATEGIES

1. PROCEDURE OUTPUT

ILO A & ILO B:

Output:

Screenshot

```cpp
#include <iostream>
#include <string>
#include <thread>

void print(int n, const std::string &str) {
    std::cout << "Printing integer: " << n << std::endl;
    std::cout << "Printing string: " << str << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
    return 0;
}
```

Output:

```
Printing integer: 10
Printing string: T.I.P.
```

Analysis

De Leon	The code has a function that accepts an integer to a string. Inside the main a new concurrent thread execution, it passes the function "print"and arguments 10 and "T.I.P.". Because only one thread (t1), the output will consistently print the exact order.
Mendoza	This code teaches us how to create and execute a

	basic thread in C++ using the <thread> library. It demonstrates passing multiple arguments (10 and "T.I.P.") to a function pointer (print) upon spawning a worker thread (t1).

Table 13-1. Simple One-Threaded Example

Screenshot

```cpp
#include <iostream>
#include <vector>
#include <thread>

void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;
}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };
    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }
    return 0;
}
```

Outputs:

De Leon, Benj Federic	PS C:\Users\Benj Federic De Leon\Documer s\CPE21S1\CPE 010-CPE21S1\HoA13p1\" ; if 3 : Engineers1 : Competent 0 : T.I.P. 2 : Computer

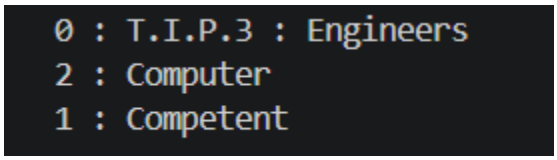
	Mendoza, Ryo	
Analysis		
	De Leon	The code dynamically spawns a vector of parallel threads to print a list of strings concurrently and safely joins them before exiting.
	Mendoza	The code demonstrates multithreading in C++ using std::thread. It creates four threads to print the strings stored in a vector. The join() function ensures all threads finish before the program terminates. Since threads run concurrently, the output order may vary each time the program runs, as shown in our output.

Table 13-2. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Part A: Demonstrate an understanding of parallelism, concurrency, and multithreading in C++ by answering the given questions.

Questions and Answers		
1. Write a definition of multithreading and its advantages/disadvantages		
	De Leon	Multithreading is a technique where one program runs several threads at the same time, each doing its own task while sharing the same memory. Its advantages are faster performance on multi-core CPUs, a responsive program, and easy data sharing between threads. Its disadvantages are race conditions, harder debugging, and the extra time and memory needed to create threads.
	Mendoza	Multithreading is a programming technique that allows multiple threads to execute within a single process. Its advantages include improving program responsiveness, allowing multiple tasks to run concurrently, and improving resource utilization. However, it can also cause race conditions when threads access shared data, make programs

<table>
<tr><td></td><td>
<table>
<tr><td></td><td>more difficult to debug, and require additional resources to manage threads.</td></tr>
</table>
</td></tr>
<tr><td>2. Rationalize the use of multithreading by providing at least 3 use-cases.</td><td>
<table>
<tr><td>De Leon</td><td>1. Smooth Apps: One thread downloads files in the background while another thread keeps buttons working so the app never freezes.
2. Busy Websites: A server gives each visitor their own thread so thousands of people can use the site at the exact same time.
3. Speedy Tasks: A video app splits a big job among many threads so all computer brains (cores) work together to finish it fast.</td></tr>
<tr><td>Mendoza</td><td>Multithreading is useful in different applications. In gaming, it allows graphics, sound, and game logic to be handled concurrently. In web servers, it helps process multiple client requests at the same time. In file downloading, it allows multiple files to be downloaded or processed while keeping the application responsive.</td></tr>
</table>
</td></tr>
<tr><td>3. Differentiate between parallelism and concurrency.</td><td>
<table>
<tr><td>De Leon</td><td>Concurrency means several tasks overlap in time. They may only take turns on one core, and they can share memory. Parallelism means tasks truly run at the same instant on different cores. Concurrency is about handling many tasks. Parallelism is about doing many tasks at once.</td></tr>
<tr><td>Mendoza</td><td>Concurrency occurs when multiple tasks make progress during overlapping periods, even if they take turns using a single CPU core. In contrast, parallelism occurs when multiple tasks execute at the same time,</td></tr>
</table>
</td></tr>
</table>

	typically using multiple CPU cores. For example, switching between a music player and a web browser demonstrates concurrency, while two CPU cores processing separate tasks simultaneously demonstrates parallelism.

Part B: Create C++ Code and show a solution that satisfies the given requirements below.

Code

```cpp
#include <iostream>
#include <thread>
#include <mutex>
#include <chrono>

int total = 0; // Global variable
std::mutex mtx;

void add(int value) {
    if (value == 10)
        std::this_thread::sleep_for(std::chrono::seconds(1));
    else if (value == 20)
        std::this_thread::sleep_for(std::chrono::seconds(2));
    else
        std::this_thread::sleep_for(std::chrono::seconds(3));

    std::lock_guard<std::mutex> lock(mtx);
    total += value;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

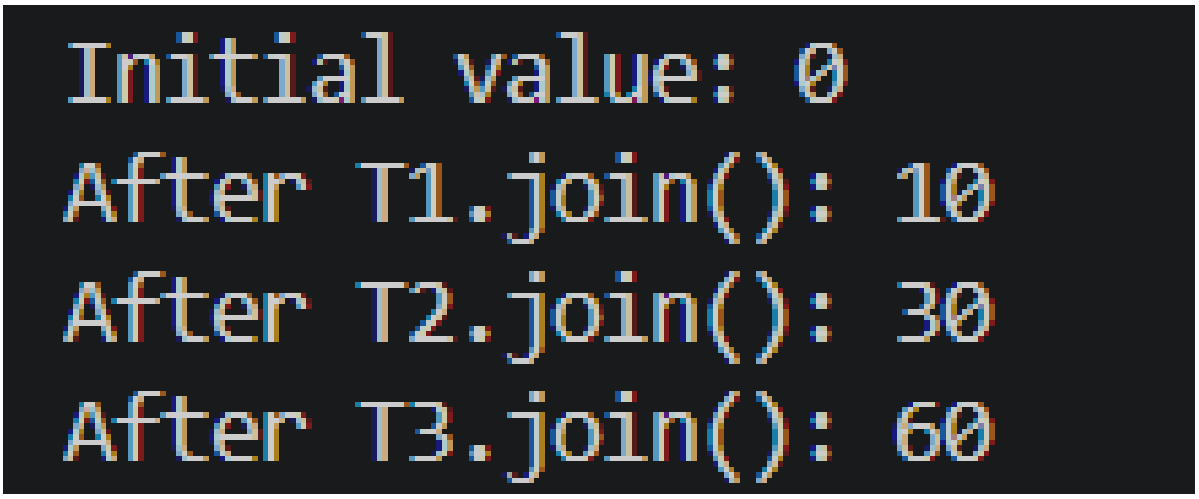
    std::cout << "Initial value: " << total << std::endl;

    T1.join();
    std::cout << "After T1.join(): " << total << std::endl;

    T2.join();
    std::cout << "After T2.join(): " << total << std::endl;

    T3.join();
    std::cout << "After T3.join(): " << total << std::endl;

    return 0;
}
```

Output


Analysis	
De Leon	The initial value is 0 because T1, T2 and T3 are still sleeping and haven't added anything yet. T1.join() waits for T1, so the total becomes 10. T2.join() makes the total 30, and T3.join() makes it 60. The three threads run together, so the whole run takes about 3 seconds. The mutex makes sure only one thread updates total at a time.
Mendoza	The program creates three threads that add 10, 20, and 30 to the global variable. Each join() waits for its corresponding thread to finish. The mutex protects the shared variable from simultaneous access. The output demonstrates how multithreading and join() affect the program's execution.

Part C: Use multi-threading with one of the algorithms previously developed in the course; provide an analysis of the result.

Code

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <chrono>

void merge(std::vector<int> &a, int l, int m, int r) {
    std::vector<int> tmp;
    int i = l, j = m + 1;
    while (i <= m && j <= r) tmp.push_back(a[i] <= a[j] ? a[i++] : a[j++]);
    while (i <= m) tmp.push_back(a[i++]);
    while (j <= r) tmp.push_back(a[j++]);
    for (int k = 0; k < (int)tmp.size(); k++) a[l + k] = tmp[k];
}

void mergeSort(std::vector<int> &a, int l, int r) {
    if (l >= r) return;
    int m = (l + r) / 2;
    mergeSort(a, l, m);
    mergeSort(a, m + 1, r);
    merge(a, l, m, r);
}

void parallelMergeSort(std::vector<int> &a, int l, int r) {
    int m = (l + r) / 2;
    std::thread left(mergeSort, std::ref(a), l, m);   //left half in a new thread
    mergeSort(a, m + 1, r);                           //right half in main thread
    left.join();
    merge(a, l, m, r);
}

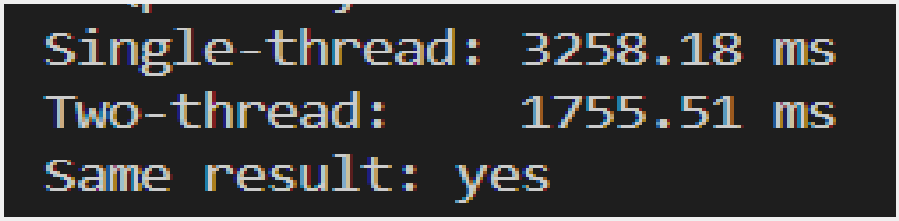
int main() {
    const int N = 2000000;
    std::vector<int> a(N);
    for (int &x : a) x = std::rand();
    std::vector<int> b = a;

    auto t0 = std::chrono::steady_clock::now();
    mergeSort(a, 0, N - 1);
    auto t1 = std::chrono::steady_clock::now();
    parallelMergeSort(b, 0, N - 1);
    auto t2 = std::chrono::steady_clock::now();

    std::cout << "Single-thread: " << std::chrono::duration<double, std::milli>(t1 - t0).count() << " ms" << std::endl;
    std::cout << "Two-thread:    " << std::chrono::duration<double, std::milli>(t2 - t1).count() << " ms" << std::endl;
    std::cout << "Same result: " << (a == b ? "yes" : "no") << std::endl;
    return 0;
}
```

Output

De Leon	Single-thread: 3264.11 ms Two-thread: 1170.88 ms Same result: yes

Mendoza	

Analysis	
De Leon	Merge sort splits the array into two independent halves, so each half can be sorted on its own thread. The left half runs in a new thread and the right half runs in main. join() waits for the left half before the final merge.
Mendoza	Two threads were about 1.9 times faster, not a full 2 times. The final merge must wait for both halves, so it can't be split between threads. Creating a thread also adds a small cost. This only pays off for large arrays. For small ones, one thread is just as fast or faster.

Conclusion	
De Leon	I learned that std::thread runs tasks at the same time, that join() makes the program wait for a thread to finish, and that a mutex stops threads from changing shared data together. The output order of the threads changed between runs. And, the total only became reliable after each join(), and splitting merge sort across two threads made it faster.
Mendoza	This activity showed me how to create threads and how multithreading differs from running code one step at a time. The procedure showed that thread output order is unpredictable, and Part B showed how join() and a mutex control when and how shared data is updated. Part C showed that threads help most when the work splits into independent parts. I did well on understanding the concepts, but I need more practice with thread safety and with measuring performance.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Claude

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

COMPUTER ENGINEERING

QUEZON CITY

Purpose of Use: Code assistance, explanation of concepts, and drafting analysis and conclusion text.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Claude was used to help write and test the C++ code and to draft the missing analysis, answers and conclusions. We reviewed and ran the code and edited the text before submitting

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.